Mini libreria per sistemi lineari

Progetto 1 Bis - Metodi del calcolo scientifico

Sabrina Guidotti
Matricola 865858
sguidotti2@campus.unimib.it

Indice

1 Implementazione in Python della libreria **1**
- 1.1 Struttura della Libreria . . . 1
- 1.2 IterativeMethodBase . . . 2
  - 1.2.1 Costruttore, setter e getter . . . 2
  - 1.2.2 Altri metodi di supporto . . . 3
  - 1.2.3 Metodo solve . . . 5
- 1.3 JacobiMethod . . . 5
- 1.4 GaussSeidelMethod . . . 5
- 1.5 GradientMethod . . . 7
- 1.6 ConjugateGradientMethod . . . 7

2 Test svolti **9**
- 2.1 Modalità test . . . 9
- 2.2 spa1 . . . 9
  - 2.2.1 Risultati . . . 11
- 2.3 spa2 . . . 13
  - 2.3.1 Risultati . . . 14
- 2.4 vem1 . . . 17
  - 2.4.1 Risultati . . . 18
- 2.5 vem2 . . . 20
  - 2.5.1 Risultati . . . 21

3 Conclusione **24**

Capitolo 1

Implementazione in Python della libreria

1.1 Struttura della Libreria

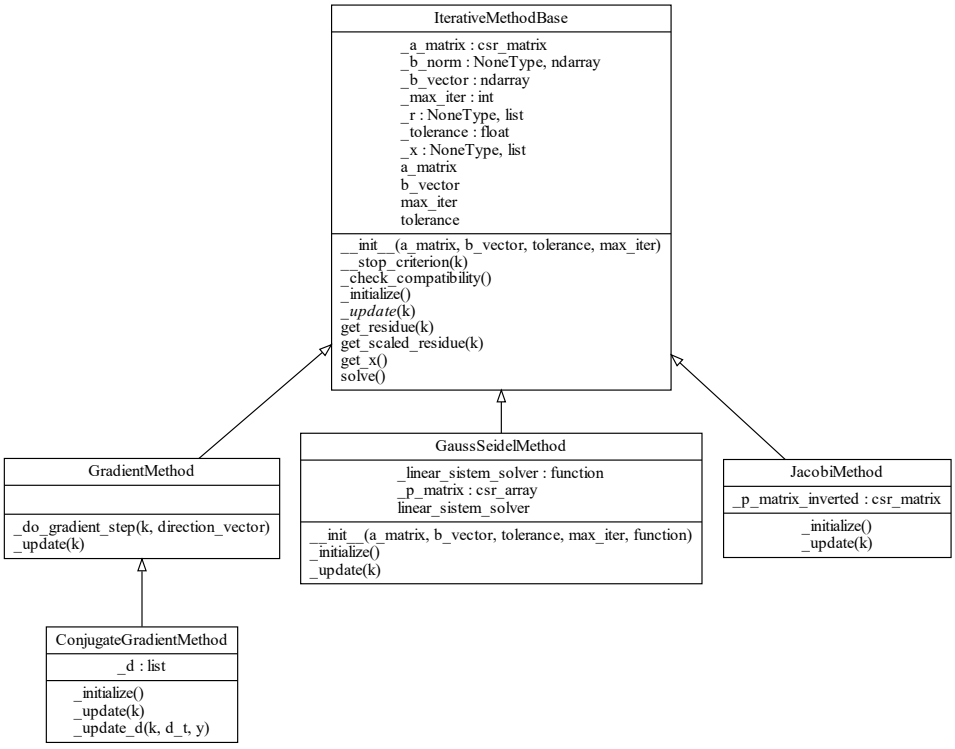


Figura 1.1.1: Struttura delle classi

Per implementare questa libreria per la risoluzione di sistemi lineare iterativi è stato utilizzato il linguaggio Python, utilizzando la libreria SciPy come libreria open-source per la gestione della struttura dati e le operazioni elementari fra matrici. E' stato adottato il paradigma di programmazione

orientata agli oggetti e seguendo il principio DRY (Don't Repeat Yourself) per mantenere il codice pulito ed efficiente.

Nella figura in alto è riportato il diagramma UML delle classi (approssimativo). Di seguito, una breve spiegazione delle notazioni utilizzate:

- **Classi**: rappresentate da rettangoli suddivisi in tre sezioni. La sezione superiore contiene il nome della classe, quella centrale elenca gli attributi e quella inferiore elenca i metodi.
- **Attributi**: mostrati nella seconda sezione del rettangolo, nel formato `nome: tipo`. Un underscore (_) davanti al nome indica che l'attributo è protetto e significa che dovrebbe essere accessibile solo all'interno della classe e dalle sue sottoclassi. Nelle ultime posizioni sono mostrate le property, il modo in cui sono implementati i getter e i setter in Python. In realtà, Python non è un linguaggio tipizzato staticamente, quindi i tipi delle variabili vengono determinati solo durante l'esecuzione.
- **Metodi**: indicati nella terza sezione del rettangolo, nel formato `nomeMetodo(parametri)`. Anche in questo caso, un underscore (_) indica che il metodo è protetto, due underscore (__) indica che il metodo è privato, mentre __init__ è il costruttore in Python. I metodi astratti, rappresentati in corsivo, sono definiti ma non implementati nella classe base e devono essere sovrascritti nelle sottoclassi.
- **Ereditarietà**: rappresentata da una freccia vuota che punta dalla classe derivata alla classe base, indicando che la classe derivata eredita gli attributi e i metodi della classe base.

1.2 IterativeMethodBase

1.2.1 Costruttore, setter e getter

Innanzitutto viene definita una classe astratta, `IterativeMethodBase`, che rappresenta una base per i metodi iterativi utilizzati per risolvere sistemi lineari iterativi della forma $A\mathbf{x} = \mathbf{b}$.

```
class IterativeMethodBase(ABC):
    def __init__(self, A, b, Tol, max_iter=20000):
        self.A = A
        self.b = b
        self.Tol = Tol
        self.max_iter = max_iter
        self._x = None
        self._r = None
        self._b_norm = None
```

Il metodo `__init__` è un costruttore. Viene chiamato quando si crea un nuovo oggetto della classe. Prende come argomenti la matrice A, il vettore b, la tolleranza e il numero massimo di iterazioni, con un valore predefinito di 20000 per il numero massimo di iterazioni. Vengono anche settati a None _x e _r, che saranno delle liste contenenti il vettore delle soluzioni e il residuo per le varie iterazioni k (saranno inizializzati prima dell'esecuzione del metodo). Viene anche inizializzato _b_norm, che conterrà la norma del vettore b, che sarà calcolata in seguito. Queste inizializzazioni vengono fatte per evitare errori di variabili non definite in metodi che possono essere chiamati prima del metodo solve.
La classe include getter e setter per gli attributi `A`, `b`, `Tol`, e `max_iter`, che sono usati nel costruttore e possono essere chiamati successivamente. I setter effettuano delle validazioni quando vengono chiamati. Sono utilizzati i decoratori `@property` per definire i getter e i decoratori `@<nome_property>.setter` per definire un metodo setter.

```
    @A.setter
    def A(self, value):
        value = sp.csr_matrix(value)
        if value.shape[0] != value.shape[1]:
            raise ValueError("Matrix A must be square.")
        if not ((value - value.T).nnz == 0 and all(spla.eigs(value, k=1,
            which='SM')[0] > 0)):
            raise ValueError("Matrix A must be symmetric and positive definite.")
        self._A = value
```

Per gestire la matrice A, prima di tutto il valore in input viene convertito in una matrice compressa nel formato CSR (Compressed Sparse Row), poiché è scritto nelle note della documentazione che è il formato più ottimizzato per lo svolgimento del prodotto matrice-vettore (tuttavia, la differenza nei tempi non è significativa).

Successivamente, viene verificato se il numero di righe della matrice è diverso dal numero di colonne; se lo sono, l'algoritmo solleverà un errore poiché la matrice non è quadrata. Dopodiché, viene controllato se il risultato della sottrazione tra la matrice originale e la sua trasposta ha zero elementi diversi da zero. In questo caso, la matrice viene considerata simmetrica. Infine, vengono verificati tutti gli autovalori della matrice per assicurarsi che siano maggiori di zero, confermando che la matrice è definita positiva.

Per gli altri parametri vengono effettuati dei controlli sul tipo in input e si verifica che la tolleranza e il numero di iterazioni siano positivi.

1.2.2 Altri metodi di supporto

```
    def get_x(self):
        return self._x

    def get_residue(self, k):
        if self._r is None:
            raise ValueError("Use the solve method before calling get_residue or
                get_scaled_residue.")
        if len(self._r) == len(self._x) - 1:
            self._r.append(self.b - self.A @ self._x[k])
        elif len(self._r) < k:
            raise ValueError("Invalid iteration number.")
        return self._r[k]

    def get_scaled_residue(self, k):
        return np.linalg.norm(self.get_residue(k)) / self._b_norm

    def __stop_criterion(self, k):
        return self.get_scaled_residue(k) < self.Tol
```

Questi sono alcuni metodi non protetti della classe (escluso l'ultimo), che oltre ad essere utilizzati all'intero della classe stessa, possono essere chiamati dall'esterno tramite l'oggetto, in seguito al metodo `solve`, per ottenere informazioni di debug o statistiche sulla procedura iterativa.

- `get_x`: restituisce l'intera lista delle soluzioni ottenute nelle diverse iterazioni dell'algoritmo iterativo. Questo metodo è utile per visualizzare la progressione del metodo risolutivo e per

contare le epoche.

- `get_residue`: restituisce il residuo per l'iterazione `k`. In particolare, se il residuo per la soluzione $\mathbf{x}^{(k)}$ con `k` uguale all'ultima iterazione non è stato calcolato, il metodo lo calcolerà; altrimenti, per motivi di efficienza, il valore calcolato in precedenza viene recuperato da una lista (poiché il residuo è necessario per diversi metodi risolutivi). Se `k` è troppo grande, verrà restituito un errore.

$$\texttt{Residuo per l'iterazione k}: \mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}$$

- `get_scaled_residue`: restituisce il residuo scalato per l'iterazione `k` specificata come input. Viene utilizzata la norma di $\mathbf{b}$ calcolata nel metodo `_initialize`, per evitare di calcolarla più volte inutilmente. A differenza del residuo, questo valore non viene salvato, poiché la funzione è utilizzata all'interno della classe solo come criterio di arresto.

$$\texttt{Residuo scalato per l'iterazione k}: \frac{\|\mathbf{b} - A\mathbf{x}^{(k)}\|}{\|\mathbf{b}\|} = \frac{\|\mathbf{r}^{(k)}\|}{\|\mathbf{b}\|}$$

- `__stop_criterion`: ritorna true se il residuo scalato per l'iterazione `k` è minore della tolleranza, false altrimenti.

```
def _check_compatibility(self):
    if self.A.shape[0] != self.b.shape[0]:
        raise ValueError("Matrix A and vector b are not compatible.")

def _initialize(self):
    self._check_compatibility()
    self._b_norm = np.linalg.norm(self.b)
    self._x = [np.zeros(self.b.shape)]
    self._r = []

@abstractmethod
def _update(self, k):
    pass
```

Questi sono altri metodi presenti nella classe:

- `_check_compatibility`: Verifica che le matrici siano compatibili tra di loro, in particolare che abbiano lo stesso numero di righe.

- `_initialize`: Chiama il metodo `_check_compatibility` per svolgere i controlli necessari. Inizializza la lista dei residui `_r` a una lista vuota e la lista `_x`, che conterrà i vari $\mathbf{x}^{(k)}$, a una lista contenente un vettore colonna di 0 con la stessa forma di `b`. Viene anche calcolata la sua norma, che servirà ogni volta che viene calcolato il residuo scalato. Anche in questo caso può essere fatto un overriding dalle classi figlie, per aggiungere inizializzazioni aggiuntive da svolgere prima di chiamare il metodo solve. Questo metodo è stato diviso dal costruttore, poiché potenzialmente il metodo solve potrebbe essere chiamato più volte e queste strutture devono essere inizializzate ogni volta (inoltre l'utente potrebbe utilizzare i setter per variare gli attributi in input, ad esempio potrebbe essere ripetuta l'esecuzione variando la tolleranza senza bisogno di ri-allocare gli altri parametri).

- `_update`: Metodo astratto che deve essere implementato dalle sottoclassi. Il suo scopo è definire come $\mathbf{x}^{(k+1)}$ viene modificato nello svolgimento del metodo iterativo.

1.2.3 Metodo solve

```
    def solve(self):
        self._initialize()
        k = 0
        while not self._stop_criterion(k):
            self._update(k)
            k += 1
            if k >= self.max_iter:
                raise RuntimeError("Non-convergence after maximum Numero iterazioni")
        return self._x[k]
```

Questo metodo costituisce il nucleo della libreria: è il metodo che deve essere chiamato per qualsiasi delle classi figlie al fine di risolvere il sistema lineare. Inizialmente, esegue le inizializzazioni chiamando il metodo `_initialize`, come precedentemente spiegato. Viene definita una variabile `k` inizializzata a zero, che rappresenta il numero di iterazioni, e viene incrementata nel ciclo successivo. La condizione di terminazione del ciclo `while` è data dal criterio di arresto `_stop_criterion`, (quindi si uscirà dal ciclo quando il residuo scalato è inferiore alla tolleranza specificata). All'interno del ciclo, viene chiamato il metodo `_update`, che verrà implementato dalle classi figlie per definire $\mathbf{x}^{(k+1)}$. Viene inoltre verificato il numero di epoche: se viene superato il limite massimo, viene generato un errore.

1.3 JacobiMethod

```
class JacobiMethod(IterativeMethodBase):
    def _initialize(self):
        super()._initialize()
        p_matrix = sp.diags(self.A.diagonal())
        self._p_matrix_inverted = spla.inv(p_matrix)

    def _update(self, k):
        self._x.append(self._x[k] + self._p_matrix_inverted @ self.get_residue(k))
```

Questa è l'implementazione del metodo di Jacobi, eredita dalla classe 'IterativeMethodBase'. Inizialmente, il metodo `_initialize` viene sovrascritto (override) per includere il calcolo della matrice P^{-1}, che deve essere calcolata una sola volta. Per ottenere P^{-1}, viene estratta la diagonale della matrice, e successivamente viene creata una matrice sparsa contenente solo la diagonale, mentre tutti gli altri elementi sono impostati a zero. Tale matrice viene poi invertita e salvata nella variabile `_p_matrix_inverted`. Nel metodo `_update`, viene aggiunto un nuovo elemento alla lista `_x` in posizione $k+1$, calcolato come segue:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathsf{P}^{-1}\mathbf{r}^{(k)}$$

1.4 GaussSeidelMethod

```
class GaussSeidelMethod(IterativeMethodBase):
    def __init__(self, A, b, Tol, max_iter=20000, function="spsolve_triangular"):
        super().__init__(A, b, Tol, max_iter)
        self.linear_sistem_solver = function
```

```
    @property
    def linear_sistem_solver(self):
        return self._linear_sistem_solver

    @linear_sistem_solver.setter
    def linear_sistem_solver(self, value):
        if value == "spsolve_triangular":
            self._linear_sistem_solver = spla.spsolve_triangular
        else:
            self._linear_sistem_solver = forward_substitution

    def _initialize(self):
        super()._initialize()
        self._p_matrix = sp.tril(self.A).tocsr()

    def _update(self, k):
        y = self.linear_sistem_solver(self._p_matrix, self.get_residue(k))
        self._x.append(self._x[k] + y)
```

Questa è l'implementazione del metodo di Gauß-Seidel, che eredita dalla classe `IterativeMethodBase`. In questa classe viene effettuato l'override del metodo `_initialize` per includere il calcolo della matrice P. La matrice P è ottenuta estraendo la matrice triangolare inferiore (mantenuta sempre in formato sparso), che viene poi convertita nel formato CSR (Compressed Sparse Row).

Tuttavia, la matrice P, essendo triangolare inferiore, non è facilmente invertibile, a differenza della matrice contenente solo la diagonale di Jacobi. Ciò nonostante, una matrice triangolare inferiore è facilmente risolvibile come sistema lineare, utilizzando la sostituzione in avanti. Infatti, si ha che:

$$\mathbf{y} = \mathsf{P}^{-1}\mathbf{r}^{(k)} \quad \Rightarrow \quad \mathsf{P}\mathbf{y} = \mathbf{r}^{(k)}$$

Una volta calcolato $\mathbf{y}$, per trovare $\mathbf{x}^{(k+1)}$ sarà sufficiente eseguire:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{y}$$

Nel metodo `_update` viene calcolato il sistema lineare per trovare $\mathbf{y}$ e successivamente viene eseguita l'operazione per trovare $\mathbf{x}^{(k+1)}$. Come detto in precedenza, per il calcolo del sistema lineare per trovare $\mathbf{y}$ è sufficiente eseguire una sostituzione in avanti. Per fare ciò sono stati sperimentati due metodi: una funzione offerta dalla libreria SciPy, `spsolve_triangular`, che risolve sistemi lineari con matrici triangolari sparse in modo efficiente, e una funzione creata da me, che risolve un sistema lineare eseguendo una sostituzione in avanti, implementata come segue:

```
def forward_substitution(L, b):
    n = L.shape[0]
    x = np.zeros(n)

    if L[0, 0] == 0:
        raise ValueError("Invalid L matrix matrix")

    x[0] = b[0] / L[0, 0]
    for i in range(1, n):
        if L[i, i] == 0:
            raise ValueError("Invalid L matrix matrix")
        x[i] = (b[i] - L[i, :] @ x) / L[i, i]

    return x
```

In questa classe è presente un parametro aggiuntivo per selezionare quale funzione utilizzare.

1.5 GradientMethod

```
class GradientMethod(IterativeMethodBase):
    def _do_gradient_step(self, k, direction_vector):
        d_t = direction_vector.T
        y = self.A @ direction_vector
        a_k = (d_t @ self.get_residue(k)) / (d_t @ y)
        self._x.append(self._x[k] + (a_k * direction_vector))
        return d_t, y

    def _update(self, k):
        self._do_gradient_step(k, self.get_residue(k))
```

In questo caso, il metodo `_update` chiama un altro metodo, chiamato `_do_gradient_step`, che, avendo in input un certo vettore direzione e il numero di iterazione k, si occupa di eseguire il passo k nella discesa del gradiente e calcolare $\mathbf{x}^{(k+1)}$.Nel metodo del gradiente, la direzione è il residuo $\mathbf{r}^{(k)}$. Nel metodo del gradiente $\mathbf{x}^{(k+1)}$ viene calcolato nel seguente modo:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}$$

dove α_k è uguale a:

$$\alpha_k = \frac{(\mathbf{r}^{(k)})^T \mathbf{r}^{(k)}}{(\mathbf{r}^{(k)})^T A \mathbf{r}^{(k)}}$$

Tuttavia, è possibile considerare queste operazioni in modo più astratto, utilizzando un vettore direzione generico $\mathbf{d}^{(k)}$:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)}$$

dove α_k è uguale a:

$$\alpha_k = \frac{(\mathbf{d}^{(k)})^T \mathbf{r}^{(k)}}{(\mathbf{d}^{(k)})^T A \mathbf{d}^{(k)}}$$

Per implementare correttamente il metodo del gradiente, basta considerare $\mathbf{d}^{(k)}$ come $\mathbf{r}^{(k)}$, che infatti viene sempre passato come input alla funzione `_do_gradient_step`. Vedendo le operazioni in questo modo, è possibile ottenere una funzione generica e riutilizzabile, adatta anche per essere impiegata nel metodo del gradiente coniugato, applicando così il principio DRY (Don't Repeat Yourself).

1.6 ConjugateGradientMethod

```
class ConjugateGradientMethod(GradientMethod):
    def _initialize(self):
        super()._initialize()
        self._d = [self.get_residue(0)]

    def _update_d(self, k, d_t, y):
        w = self.A @ self.get_residue(k + 1)
        beta = (d_t @ w) / (d_t @ y)
        self._d.append(self.get_residue(k + 1) - beta * self._d[k])
```

```
def _update(self, k):
    d_t, y = self._do_gradient_step(k, self._d[k])
    self._update_d(k, d_t, y)
```

La classe `ConjugateGradientMethod` eredita dalla classe `GradientMethod`, poiché anch'essa utilizza `_do_gradient_step`, precedentemente definita in `GradientMethod`. In questo caso, il vettore della direzione è calcolato nel seguente modo:

$$\mathbf{d}^{(k+1)} = \mathbf{r}^{(k+1)} - \beta_k \mathbf{d}^{(k)}$$

dove β_k è uguale a:

$$\beta_k = \frac{(\mathbf{d}^{(k)})^T A \mathbf{r}^{(k+1)}}{(\mathbf{d}^{(k)})^T A \mathbf{d}^{(k)}}$$

Questo è realizzato nella funzione `_update_d`. La funzione prende come input anche il vettore direzione trasposto e $y = A\mathbf{d}^{(k)}$, poiché erano già stati calcolati in precedenza e sarebbe inefficiente ricalcolarli. È presente una variabile aggiuntiva, una lista `d`, che contiene i vari $\mathbf{d}^{(k)}$. Questa lista viene inizializzata nel seguente modo: $\mathbf{d}^{(0)} = \mathbf{r}^{(0)}$.

Inoltre, data la frequente utilizzazione del residuo in diversi punti, risulta molto utile avere una funzione come `get_residue(k)`, che si occupa di calcolare il residuo solo la prima volta e di restituire il valore calcolato nelle successive, rendendo il codice più efficiente e pulito.

Capitolo 2

Test svolti

2.1 Modalità test

I vari metodi sono stati testati su quattro matrici sparse diverse: `spa1`, `spa2`, `vem1`, e `vem2`. Le informazioni specifiche su queste matrici sono riportate di seguito. In particolare, è stata seguita la procedura standard per la validazione dei metodi iterativi e la valutazione dell'errore:

1. E' stato creato il vettore soluzione esatta x_{es} di lunghezza n (data la matrice $A \in \mathbb{R}^{n \times n}$) con tutte le entrate pari a 1.
2. Viene creata la soluzione b del sistema $\mathbf{b} = Ax$
3. Viene calcolata la soluzione x_{ap} con il metodo iterativo
4. Si considera l'errore relativo, il numero di iterazioni e il tempo di calcolo (risultato della media di 5 iterazioni, dato che i tempi tendono ad avere variazioni).

2.2 spa1

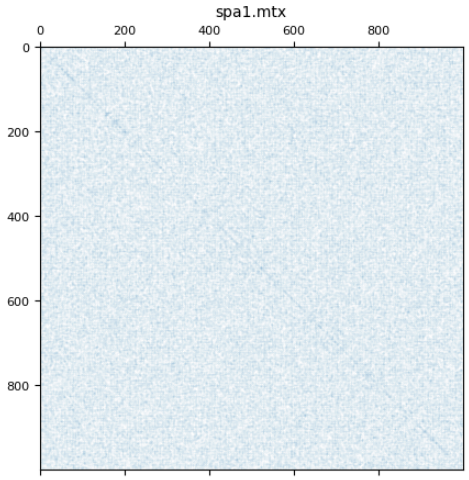


Figura 2.2.1: SPY su spa1

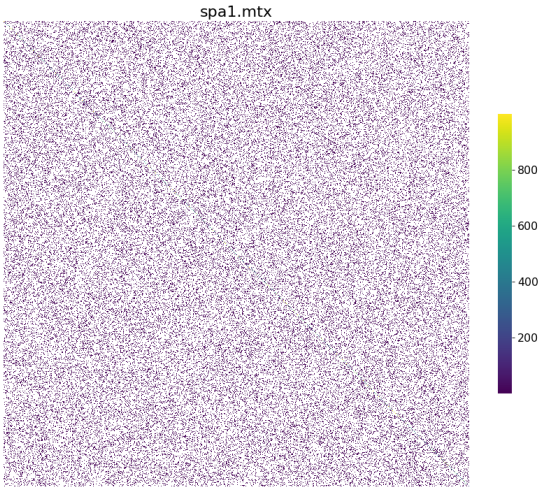


Figura 2.2.2: Heatmap di spa1

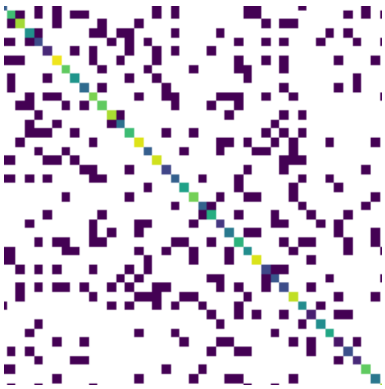


Figura 2.2.3: Dettaglio diagonale di spa1

Matrice spa1	
Rows	1000
Cols	1000
Area	1000000
Density (%)	18.243
Mean	3.2195
Std Dev	41.72
Condition Number	2048.2

Tabella 2.2.1: Analisi spa1

Ecco le informazioni riguardanti la matrice spa1. In particolare, si osserva che si tratta di una matrice sparsa con una densità di circa il 18%. Un'altra caratteristica rilevante è che gli elementi situati al di fuori della diagonale tendono ad avere valori relativamente bassi, mentre quelli posizionati sulla diagonale presentano valori molto elevati. Possiamo inoltre notare che la matrice ha un numero di condizionamento abbastanza alto.

2.2.1 Risultati

Tol	Metodo	Numero iterazioni	Tempo (s)	Errore relativo
0.0001	Jacobi	115	7.72E-02	1.77E-03
	Gauss-Seidel	9	4.78E-02	1.82E-02
	Gradient	143	3.73E-02	3.46E-02
	Conjugate Gradient	49	2.04E-02	2.08E-02
1.00E-06	Jacobi	181	1.28E-01	1.80E-05
	Gauss-Seidel	17	8.33E-02	1.30E-04
	Gradient	3577	9.70E-01	9.68E-04
	Conjugate Gradient	134	6.06E-02	2.55E-05
1.00E-08	Jacobi	247	1.66E-01	1.83E-07
	Gauss-Seidel	24	1.17E-01	1.71E-06
	Gradient	8233	2.27E+00	9.82E-06
	Conjugate Gradient	177	7.09E-02	1.32E-07
1.00E-10	Jacobi	313	2.10E-01	1.85E-09
	Gauss-Seidel	31	1.50E-01	2.25E-08
	Gradient	12919	3.69E+00	9.82E-08
	Conjugate Gradient	200	8.55E-02	1.20E-09

Tabella 2.2.2: Risultati spa1

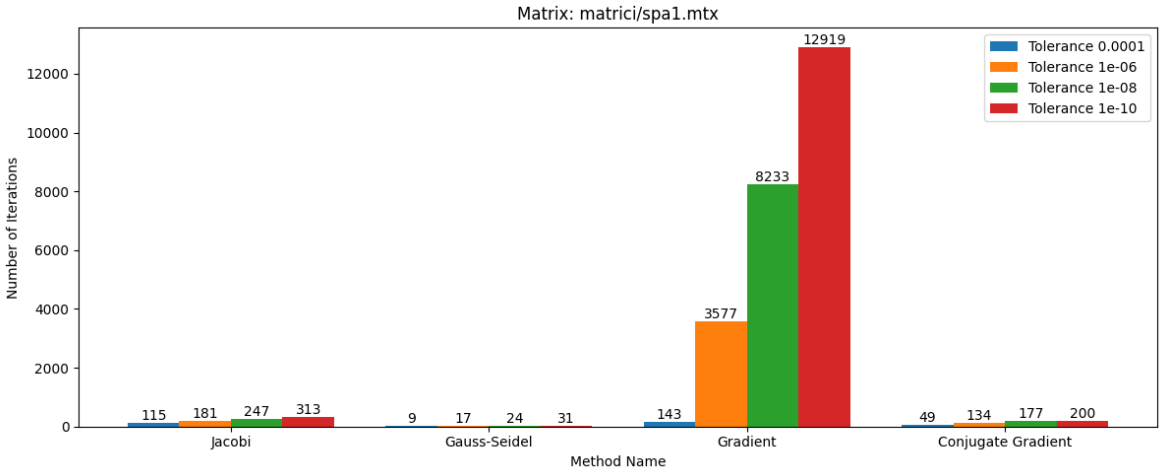


Figura 2.2.4: Numero di iterazioni per spa1

Queste sono il numero di iterazioni impiegate da ciascun metodo. In particolare, possiamo notare che il metodo del gradiente impiega un numero di epoche molto più alto rispetto a qualsiasi altro metodo. Questo è dovuto all'alto numero di condizionamento, poiché $\lambda_{min} \ll \lambda_{max}$. Geometricamente, questo significa che le linee di livello della funzione obiettivo sono fortemente allungate, rendendo il percorso di discesa più tortuoso e inefficiente. Il metodo del gradiente si muove lungo la direzione di massima discesa, ma a causa della forma ellittica delle linee di livello, i passi effettuati sono spesso molto piccoli e deviati, portando a un numero elevato di iterazioni. Questo è anche dovuto a una scelta "sfortunata" del punto di partenza, poiché un altro punto di partenza avrebbe potuto portare a un numero di iterazioni (molto) minore.

Invece, il metodo di Gauß-Seidel è quello che richiede un numero minore di iterazioni, seguito dal metodo di Jacobi. In questo caso, nonostante il metodo del Gradiente Coniugato non soffra dello stesso problema del gradiente, ha impiegato più epoche sia di Jacobi che di Gauß-Seidel.

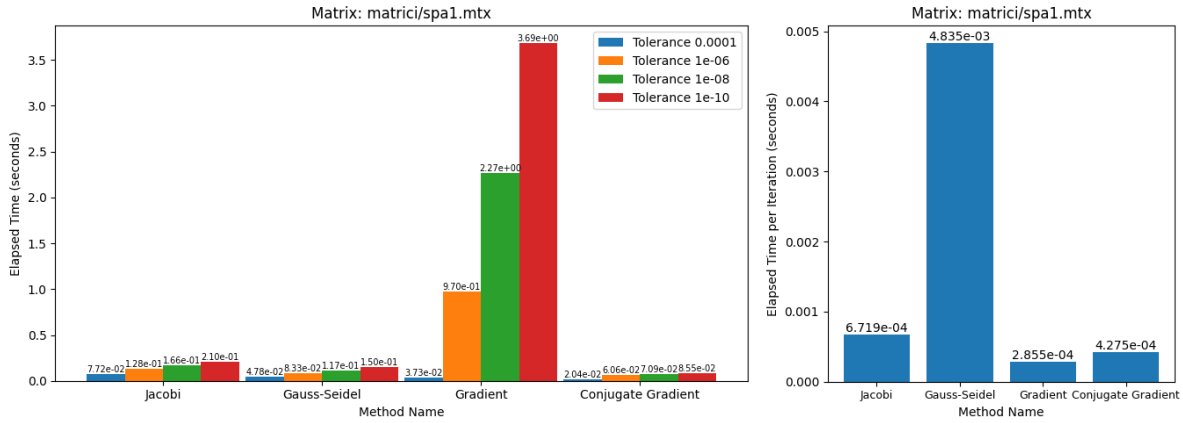


Figura 2.2.5: Tempo per spa1

Questi sono i tempi totali e per iterazione impiegati. È difficile svolgere un'analisi precisa dei tempi, dato che ci sono molti fattori in gioco, tra cui le ottimizzazioni effettuate dalla libreria per particolari operazioni e per le matrici sparse. Inoltre, il diverso stato di traffico della CPU influenza i tempi di esecuzione, portando alcuni processi ad avere tempi leggermente più elevati. Per questo motivo, i risultati visualizzati rappresentano la media di cinque prove.

I tempi tendono ad essere abbastanza consistenti con il numero di iterazioni svolte. Tuttavia, è possibile notare che il metodo di Gauss-Seidel presenta i risultati peggiori in termini di tempi. Questo è dovuto al fatto che, ad ogni iterazione, viene risolto un sistema lineare, rendendo il processo computazionalmente più intenso rispetto agli altri metodi analizzati.

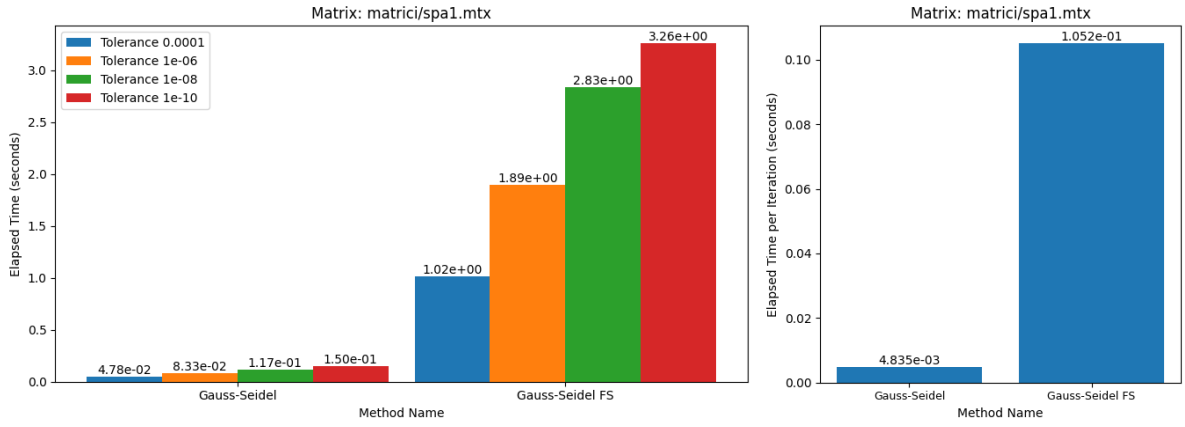


Figura 2.2.6: Tempo totale GS per spa1

Come mostrato in precedenza, è stato fatto un confronto tra l'implementazione manuale del metodo di sostituzione in avanti (FS) e il metodo offerto dalla libreria SciPy (spsolve_triangular). In termini di errore relativo e iterazioni chiaramente i risultati sono identici, tuttavia l'implementazione manuale risulta molto più lenta rispetto a quella offerta dalla libreria.

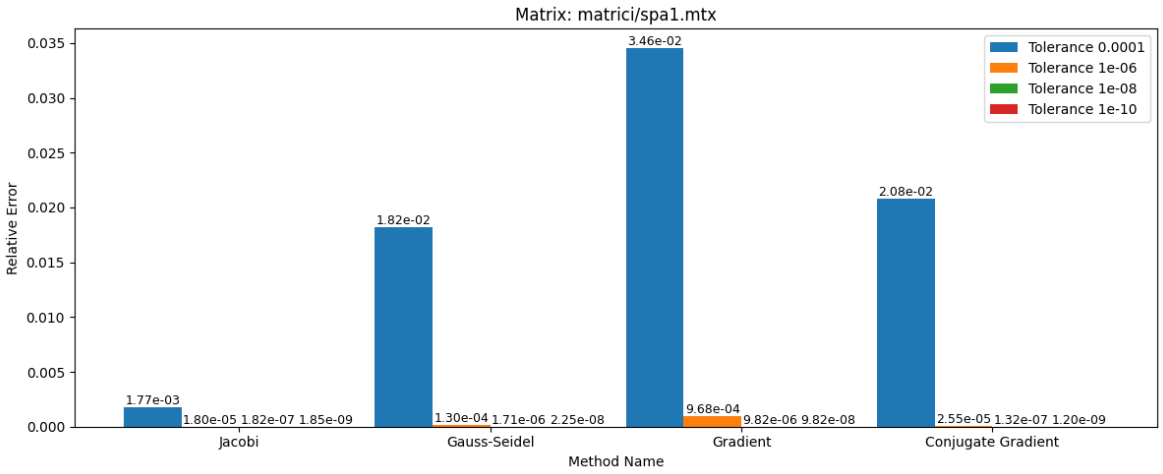


Figura 2.2.7: Errore per spa1

L'errore ovviamente diminuisce con tolleranze più basse.

2.3 spa2

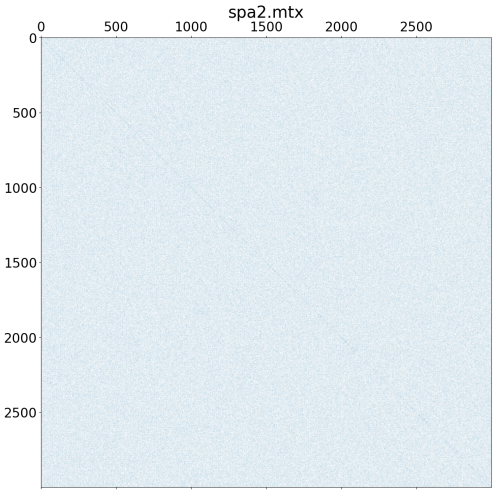


Figura 2.3.1: SPY su spa2

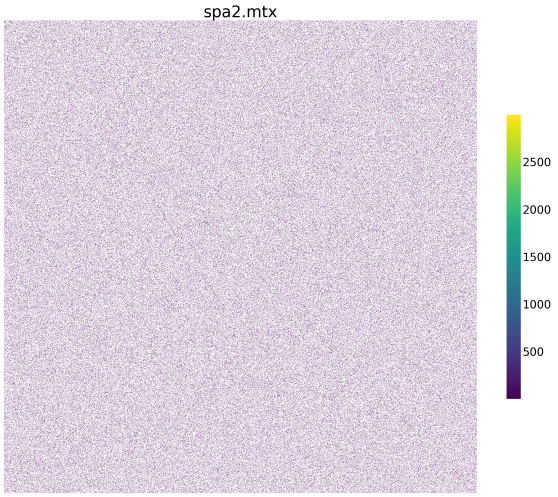


Figura 2.3.2: Heatmap di spa2

Matrice spa2	
Rows	3000
Cols	3000
Area	9000000
Density (%)	18.148
Mean	3.3427
Std Dev	74.887
Condition Number	1412.0

Tabella 2.3.1: Analisi spa2

La matrice spa2 è una versione più grande di spa1. Possono essere fatte le stesse considerazioni della matrice spa1. Una differenza è che il numero di condizionamento risulta più basso (circa $\frac{2}{3}$) di quello di spa1.

2.3.1 Risultati

Tol	**Metodo**	**Numero iterazioni**	**Tempo (s)**	**Errore relativo**
0.0001	Jacobi	36	2.74E-01	1.77E-03
	Gauss-Seidel	5	1.36E-01	2.60E-03
	Gradient	161	4.11E-01	1.81E-02
	Conjugate Gradient	42	1.71E-01	9.82E-03
1.00E-06	Jacobi	57	4.25E-01	1.67E-05
	Gauss-Seidel	8	1.69E-01	5.14E-05
	Gradient	1949	5.93E+00	6.69E-04
	Conjugate Gradient	122	4.83E-01	1.20E-04
1.00E-08	Jacobi	78	5.94E-01	1.57E-07
	Gauss-Seidel	12	2.42E-01	2.79E-07
	Gradient	5087	1.34E+01	6.87E-06
	Conjugate Gradient	196	7.46E-01	5.59E-07
1.00E-10	Jacobi	99	7.51E-01	1.48E-09
	Gauss-Seidel	15	2.91E-01	5.57E-09
	Gradient	8285	2.12E+01	6.94E-08
	Conjugate Gradient	240	9.18E-01	5.32E-09

Tabella 2.3.2: Risultati spa2

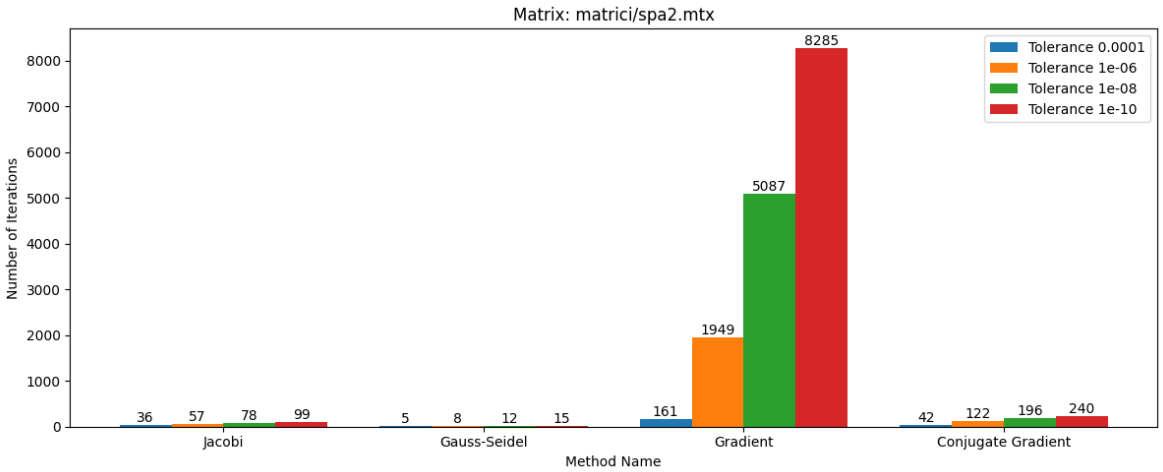


Figura 2.3.3: Numero di iterazioni per spa2

Tranne alcune eccezioni le iterazioni sembrano essere minori rispetto a quelle svolte nella risoluzione di spa1.

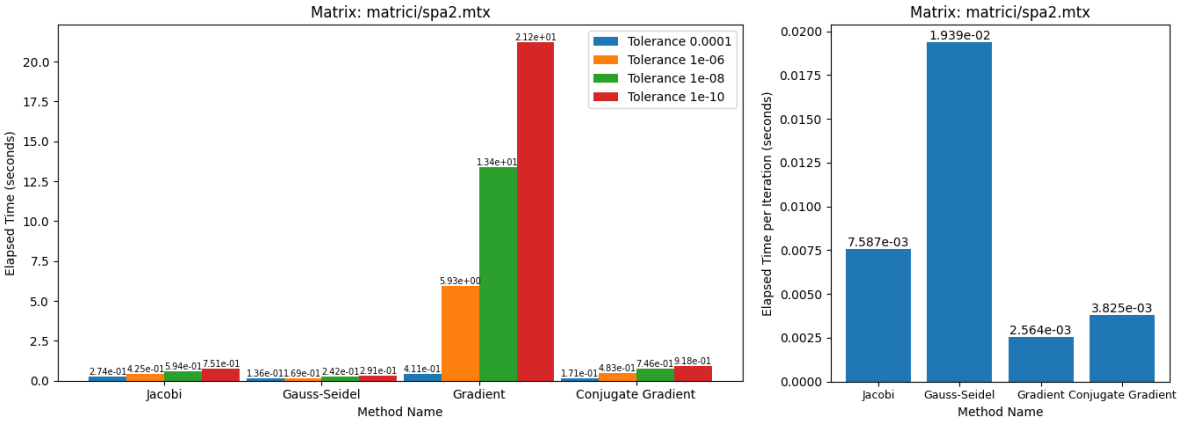


Figura 2.3.4: Tempo per spa2

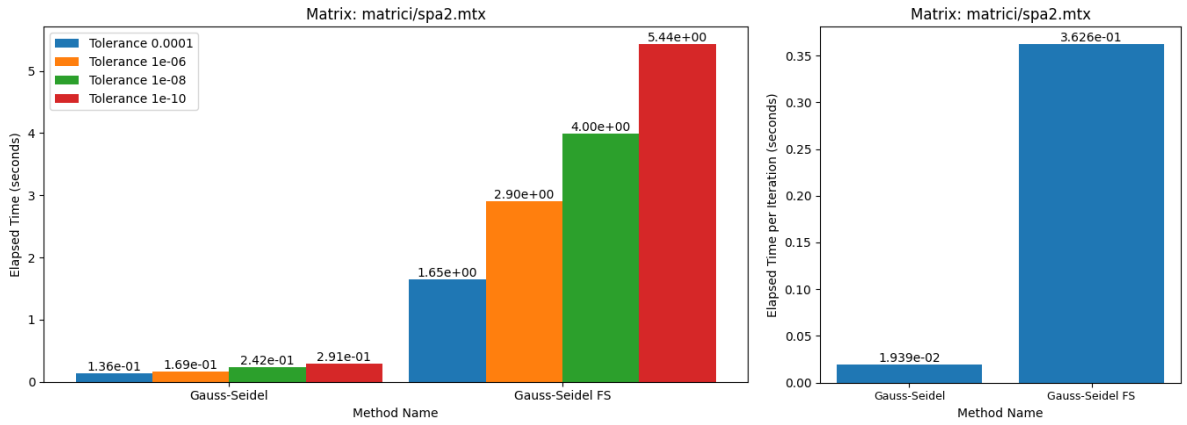


Figura 2.3.5: Tempo GS per spa2

I tempi per iterazione in questo caso sono maggiori a causa della dimensione maggiore della matrice

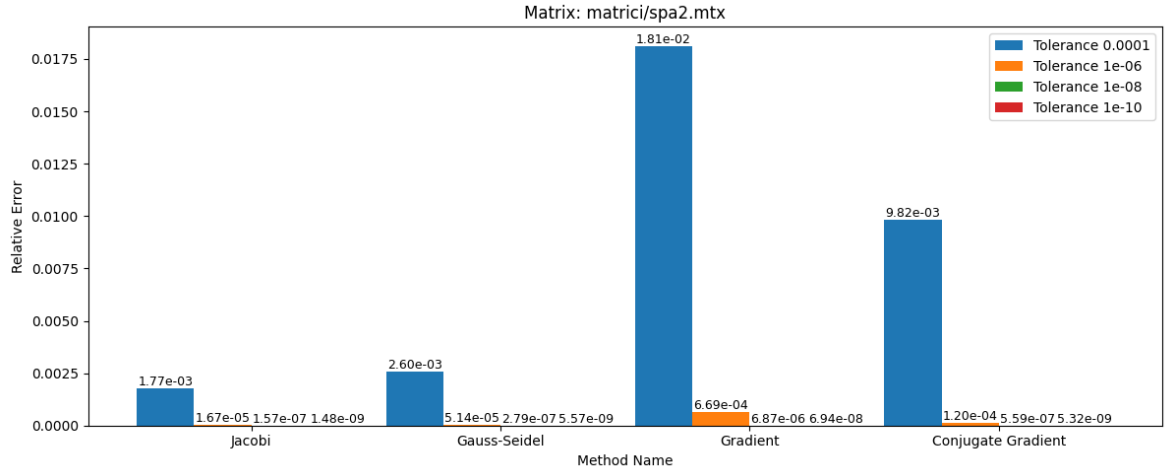


Figura 2.3.6: Errore per spa2

2.4 vem1

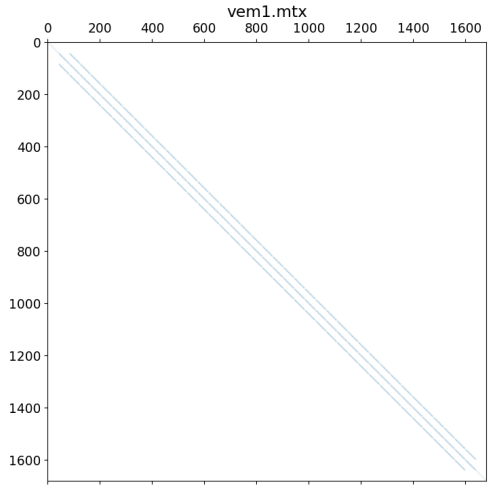


Figura 2.4.1: SPY su vem1

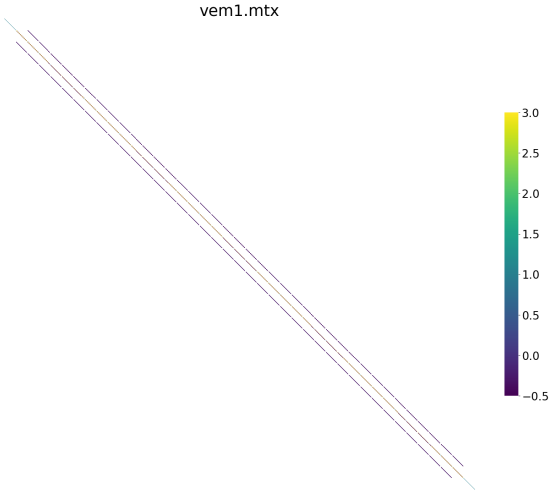


Figura 2.4.2: Heatmap di vem1

Matrice vem1	
Rows	1681
Cols	1681
Area	2825761
Density (%)	0.4737
Mean	0.0235
Std Dev	1.0825
Condition Number	324.64

Tabella 2.4.1: Analisi vem1

Questi sono i dati della matrice `vem1`. Si può notare che si tratta di una matrice significativamente diversa rispetto alle altre due. Presenta una densità molto più bassa, con i valori concentrati principalmente attorno alla diagonale principale. Questa matrice ha un numero di condizionamento inferiore alle matrici precedenti.

2.4.1 Risultati

Tol	Metodo	Numero iterazioni	Tempo (s)	Errore relativo
0.0001	Jacobi	1314	3.36E-01	3.54E-03
	Gauss-Seidel	659	4.60E+00	3.51E-03
	Gradient	890	5.18E-02	2.70E-03
	Conjugate Gradient	38	3.00E-03	4.08E-05
1.00E-06	Jacobi	2433	4.34E-01	3.54E-05
	Gauss-Seidel	1218	9.07E+00	3.53E-05
	Gradient	1612	9.04E-02	2.71E-05
	Conjugate Gradient	45	3.40E-03	3.73E-07
1.00E-08	Jacobi	3552	4.78E-01	3.54E-07
	Gauss-Seidel	1778	1.40E+01	3.52E-07
	Gradient	2336	1.38E-01	2.70E-07
	Conjugate Gradient	53	4.00E-03	2.83E-09
1.00E-10	Jacobi	4671	5.52E-01	3.54E-09
	Gauss-Seidel	2338	1.86E+01	3.51E-09
	Gradient	3058	1.88E-01	2.71E-09
	Conjugate Gradient	59	4.50E-03	2.19E-11

Tabella 2.4.2: Risultati vem1

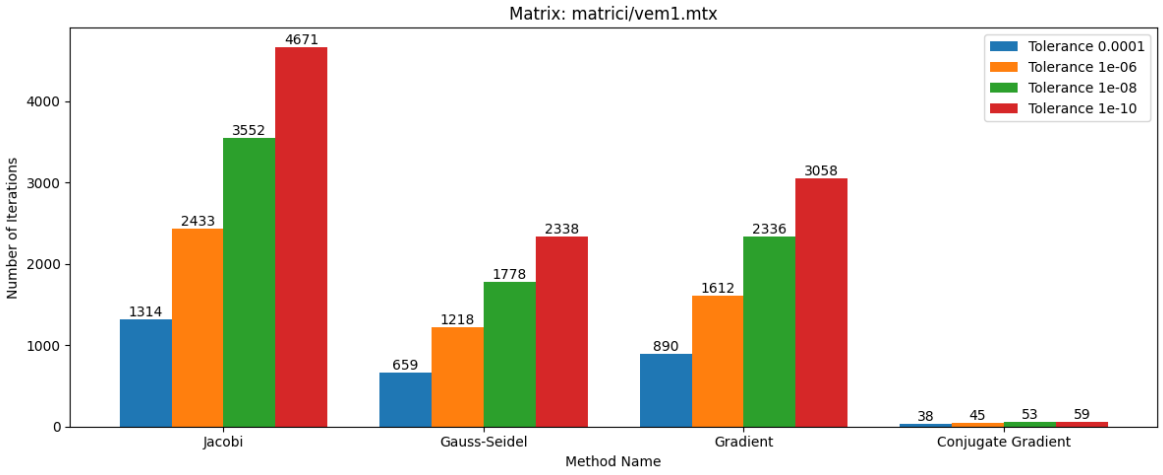


Figura 2.4.3: Numero di iterazioni per vem1

È possibile notare svariate differenze rispetto alle matrici `spa1` e `spa2`. In particolare, il metodo di Jacobi richiede un numero di epoche maggiore rispetto a tutti gli altri metodi. Al contrario, il metodo del gradiente coniugato risulta estremamente più rapido di tutti gli altri.

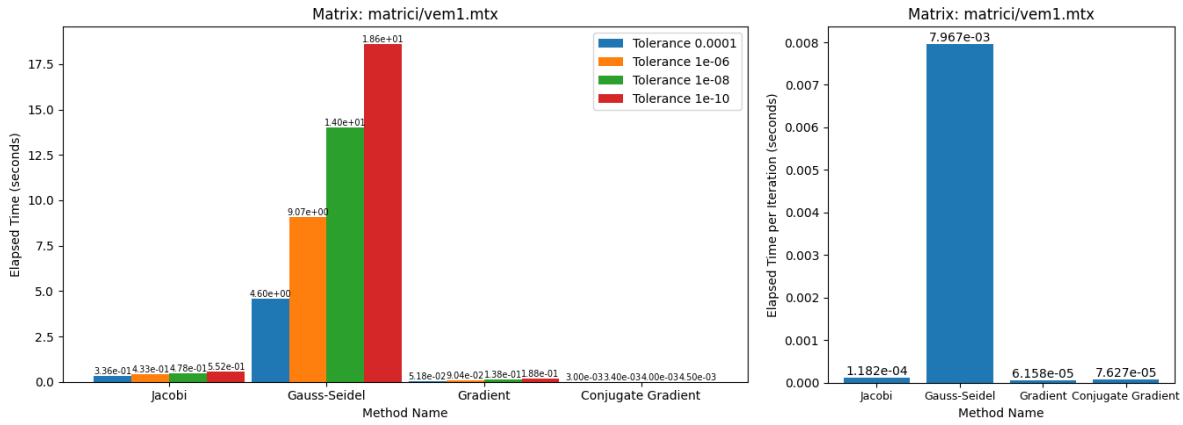


Figura 2.4.4: Tempo per vem1

In questo caso diventano evidenti i tempi aggiuntivi del metodo di Gauß-Seidel. Qui sono eseguite molte più operazioni per risolvere il sistema lineare e il tempo aggiuntivo per operazione diventa davvero significativo. Nonostante il maggior numero di iterazioni, il metodo di Jacobi risulta essere una soluzione più veloce di Gauß-Seidel in questo contesto, grazie alla sua maggiore velocità per iterazione. La combinazione della velocità di convergenza del metodo del gradiente coniugato e della velocità per operazione rende l'esecuzione complessiva molto più veloce rispetto agli altri metodi.

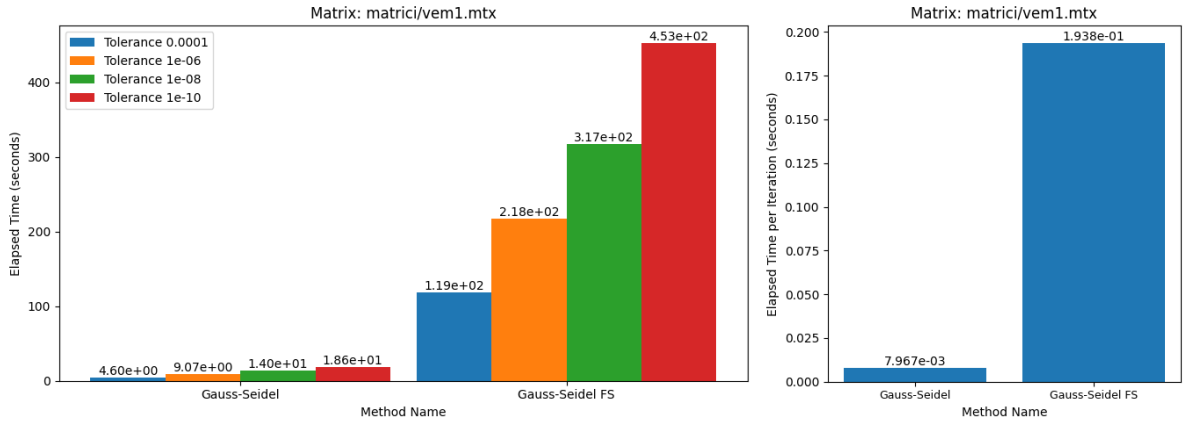


Figura 2.4.5: Tempo GS per vem1

L'algoritmo di sostituzione in avanti, a causa della sua maggiore lentezza per iterazione, comporta tempi di esecuzione considerevolmente alti.

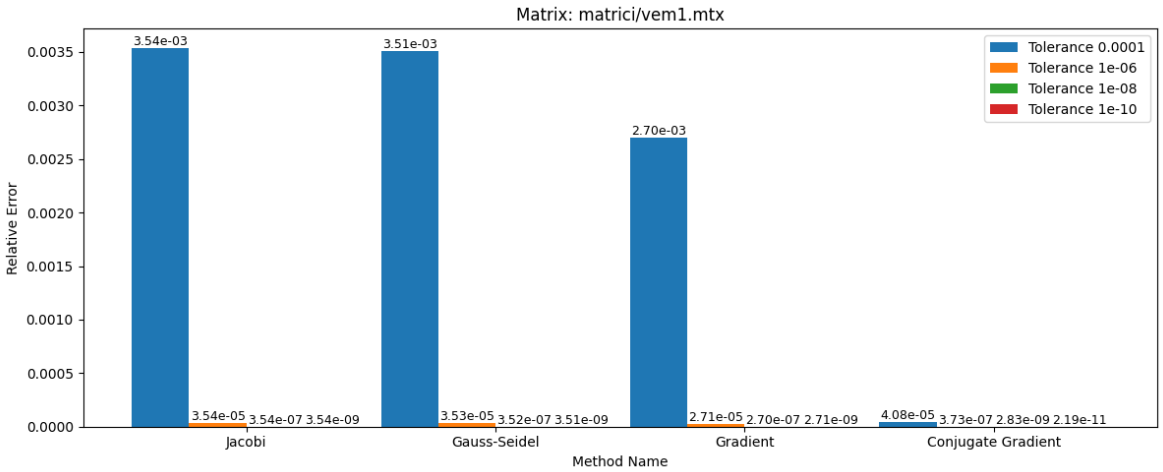


Figura 2.4.6: Errore per vem1

Il metodo del gradiente coniugato in questo caso ha portato ad avere un errore relativo molto più basso degli altri a partire dal livello di tolleranza più alto.

2.5 vem2

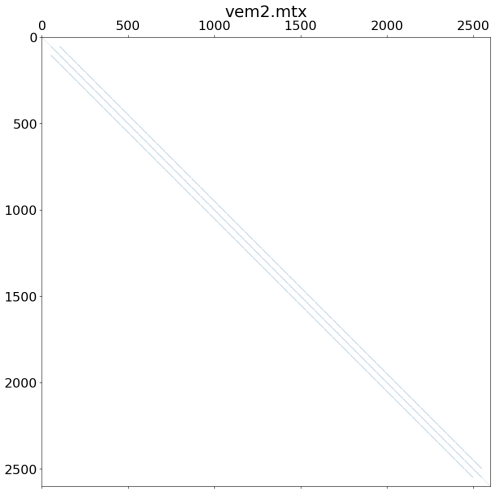


Figura 2.5.1: SPY su vem2

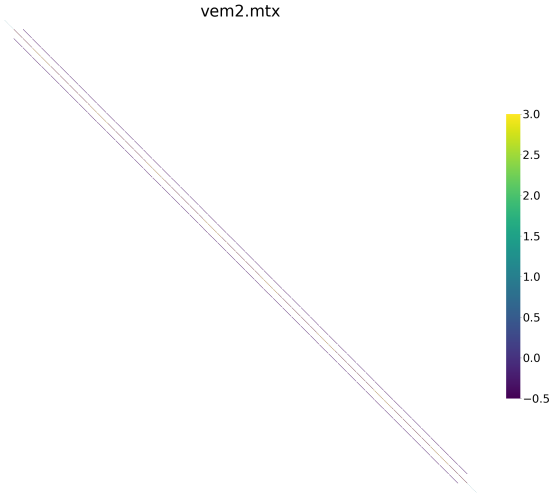


Figura 2.5.2: Heatmap di vem2

Matrice vem2	
Rows	2601
Cols	2601
Area	6765201
Density (%)	0.3137
Mean	0.0186
Std Dev	1.0794
Condition Number	507.02

Tabella 2.5.1: Analisi vem2

Anche in questo caso `vem2` si tratta di una versione più grande di `vem1`. Ha un numero di condizionamento più alto di `vem1`.

2.5.1 Risultati

Tol	Metodo	Numero iterazioni	Tempo (s)	Errore relativo
0.0001	Jacobi	1927	6.74E-01	4.97E-03
	Gauss-Seidel	965	1.07E+01	4.95E-03
	Gradient	1308	1.15E-01	3.81E-03
	Conjugate Gradient	47	6.80E-03	5.73E-05
1.00E-06	Jacobi	3676	7.39E-01	4.97E-05
	Gauss-Seidel	1840	2.11E+01	4.94E-05
	Gradient	2438	2.27E-01	3.79E-05
	Conjugate Gradient	56	7.20E-03	4.74E-07
1.00E-08	Jacobi	5425	8.32E-01	4.97E-07
	Gauss-Seidel	2714	3.21E+01	4.96E-07
	Gradient	3566	3.08E-01	3.81E-07
	Conjugate Gradient	66	9.90E-03	4.30E-09
1.00E-10	Jacobi	7174	9.73E-01	4.96E-09
	Gauss-Seidel	3589	4.21E+01	4.95E-09
	Gradient	4696	3.93E-01	3.80E-09
	Conjugate Gradient	74	1.34E-02	2.25E-11

Tabella 2.5.2: Risultati vem2

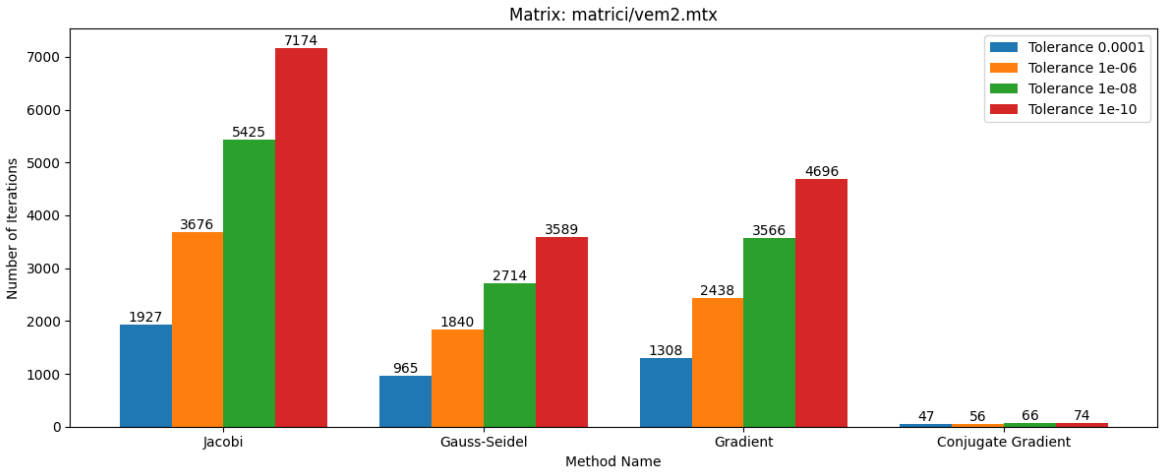


Figura 2.5.3: Numero di iterazioni per vem2

Tutti i metodi presentano un numero di iterazioni più alto, molto evidente nel caso del metodo di Jacobi.

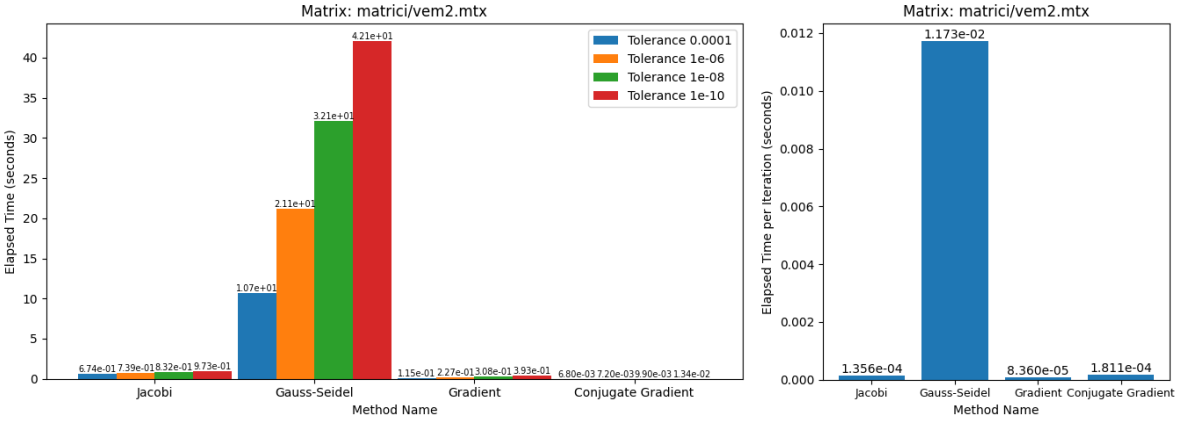


Figura 2.5.4: Tempo per vem2

I tempi risultano più alti complessivamente sia complessivamente (per il numero di iterazioni maggiori), sia nella singola iterazione (per la maggiore grandezza della matrice).

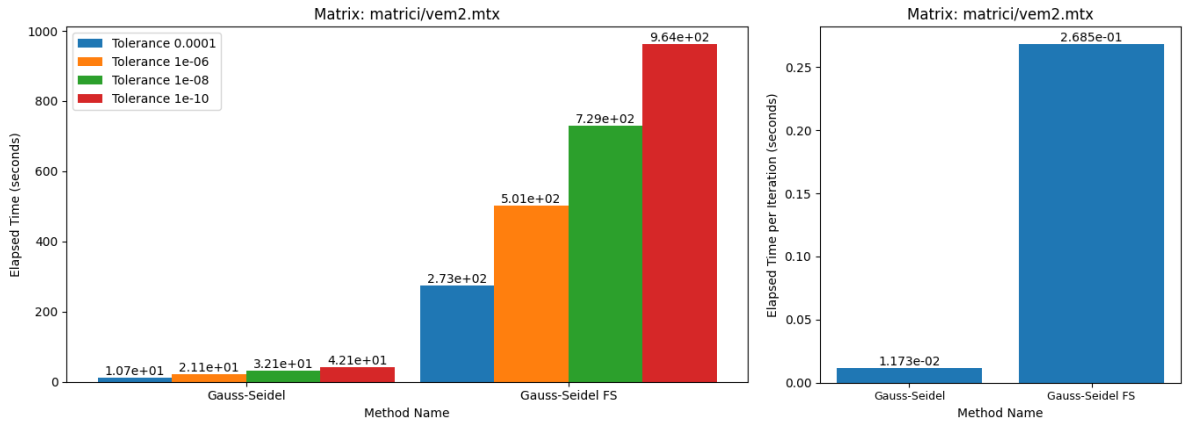


Figura 2.5.5: Tempo GS per vem2

Per i fattori considerati in precedenza il questo caso il tempo di risoluzione dell'algoritmo implementato manualmente è risultato estremamente alto (16 minuti per iterazione).

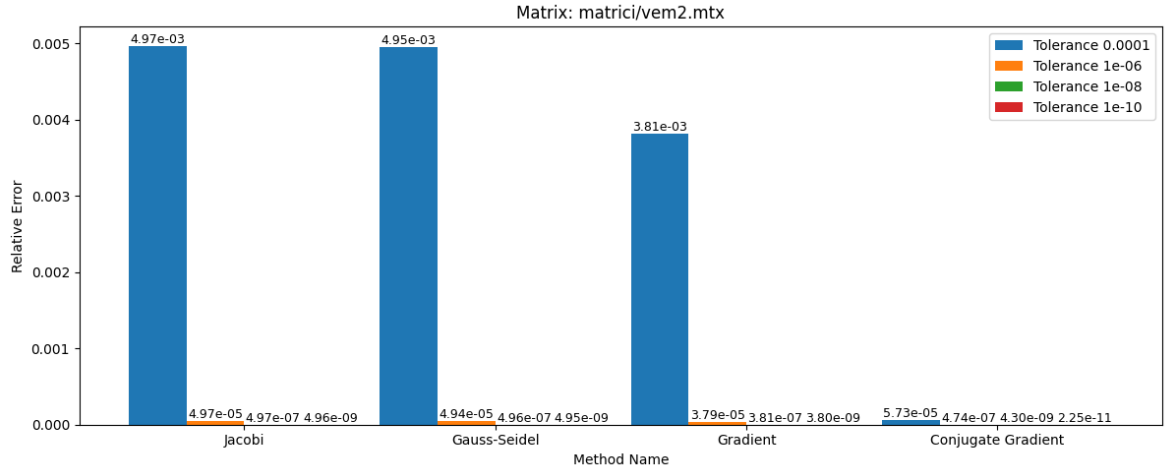


Figura 2.5.6: Errore per vem2

Capitolo 3

Conclusione

In questo lavoro è stata sviluppato una libreria Python per la risoluzione di sistemi lineari iterativi, sfruttando la libreria SciPy. La libreria segue il paradigma della programmazione orientata agli oggetti e aderisce al principio DRY per mantenere un codice pulito ed efficiente. Nello specifico implementando i metodi di: Jacobi, Gauss-Seidel, Gradiente, e Gradiente Coniugato. Ogni implementazione dei metodi è stata progettata per massimizzare la riutilizzabilità e l'efficienza.

In seguito è stata svolta un analisi tramite la libreria di quattro diverse matrici sparse (`spa1`, `spa2`, `vem1`, e `vem2`) utilizzando quattro diversi metodi iterativi implementati, testando diverse tolleranze.

E' emerso che il metodo del gradiente coniugato è stato generalmente il più efficiente in termini di numero di iterazioni. Questo risultato è particolarmente evidente per matrici `vem1` e `vem2`, dove il metodo del gradiente coniugato ha superato significativamente gli altri metodi.
Tuttavia questo metodo è stato meno efficiente (in termini di iterazioni) di Gaussß-Seidel nel caso di `spa1` e `spa2`.
D'altra parte, il metodo di Gauss-Seidel, nonostante il suo successo nella riduzione del numero di iterazioni rispetto al metodo di Jacobi, ha dimostrato di essere computazionalmente più oneroso per operazione, specialmente con matrici di dimensioni più grandi.

Inoltre, è stato evidenziato che il metodo del gradiente, a causa dell'alto numero di condizionamento delle matrici `spa1` e `spa2`, richiede un numero di iterazioni considerevolmente maggiore rispetto agli altri metodi, rendendolo meno efficiente in queste circostanze.

Infine, abbiamo confrontato anche l'implementazione manuale del metodo di sostituzione in avanti con quella offerta dalla libreria SciPy (`spsolve_triangular`). L'implementazione manuale, implementando l'algoritmo di sostituzione in avanti seguendo lo pseudocodice, ha dimostrato di essere significativamente più lenta.

In conclusione, la scelta del metodo iterativo ottimale per risolvere un sistema lineare dipende fortemente dalle caratteristiche della matrice, tra cui la densità e il numero di condizionamento. Sebbene il metodo del gradiente coniugato sia generalmente il più rapido, è importante considerare le caratteristiche specifiche del problema per ottenere il massimo delle prestazioni da un algoritmo iterativo.